

# **计算机系统导论实验报告**

**实验名称: 二进制程序分析综合实  
验 (IA-32)**

**班级: 计算 2403**

**姓名: 钟智勇**

**日期: 2026 年 1 月 7 日**

# 目录

|                                   |    |
|-----------------------------------|----|
| 1 二进制程序分析综合实验 .....               | 4  |
| 1.1 二进制程序基本组成与字符串比较 .....         | 4  |
| 1.1.1 1. 实验任务与目标 .....            | 4  |
| 1.1.2 2. 必备知识储备 .....             | 4  |
| 1.1.3 3. 实验过程记录 .....             | 4  |
| 1.1.3.1 3.1 定位核心逻辑 .....          | 4  |
| 1.1.3.2 3.2 提取目标字符串 .....         | 4  |
| 1.1.3.3 3.3 构造解答文件 .....          | 4  |
| 1.1.4 4. 遇到的问题与解决方法 .....         | 5  |
| 1.1.5 5. 心得体会 .....               | 5  |
| 1.2 浮点数表示 .....                   | 5  |
| 1.2.1 1. 实验任务与目标 .....            | 5  |
| 1.2.2 2. 完成该实验需要掌握的知识 .....       | 5  |
| 1.2.3 3. 实验过程记录 .....             | 5  |
| 1.2.4 4. 遇到的问题与解决方法 .....         | 6  |
| 1.2.5 5. 心得体会 .....               | 6  |
| 1.3 循环结构的机器级表示 .....              | 6  |
| 1.3.1 1. 实验任务与目标 .....            | 6  |
| 1.3.2 2. 完成该实验需要掌握的知识 .....       | 6  |
| 1.3.3 3. 实验过程记录（代码拆解） .....       | 6  |
| 1.3.3.1 3.1 长度与首字符检查 .....        | 6  |
| 1.3.4 3.2 循环逻辑推导 .....            | 7  |
| 1.3.4.1 3.3 手工逆推答案 .....          | 7  |
| 1.3.5 4. 遇到的问题与解决方法 .....         | 7  |
| 1.3.6 5. 心得体会 .....               | 7  |
| 1.4 选择（条件分支）结构的机器级表示 .....        | 7  |
| 1.4.1 1. 实验任务与目标 .....            | 7  |
| 1.4.2 2. 完成该实验需要掌握的知识 .....       | 8  |
| 1.4.3 3. 实验过程记录 .....             | 8  |
| 1.4.3.1 3.1 寻找跳转表与分支逻辑 .....      | 8  |
| 1.4.3.2 3.2 逆向构造目标数值 .....        | 8  |
| 1.4.3.3 3.3 构造字符串 .....           | 9  |
| 1.4.4 4. 遇到的问题与解决过程 .....         | 9  |
| 1.4.5 5. 心得体会 .....               | 9  |
| 1.5 递归过程调用 .....                  | 9  |
| 1.5.1 1. 实验任务与目标 .....            | 9  |
| 1.5.2 2. 完成该实验需要掌握的知识 .....       | 9  |
| 1.5.3 3. 实验过程记录 .....             | 9  |
| 1.5.3.1 3.1 核心逻辑定位 .....          | 9  |
| 1.5.3.2 3.2 递归函数 seek 的逆向拆解 ..... | 10 |
| 1.5.3.3 3.3 寻找正确输入 .....          | 10 |
| 1.5.4 4. 遇到的问题与解决过程 .....         | 10 |
| 1.5.5 5. 心得体会 .....               | 10 |
| 1.6 数组的存储与访问 .....                | 11 |

|         |                             |    |
|---------|-----------------------------|----|
| 1.6.1   | 1. 实验任务与目标 .....            | 11 |
| 1.6.2   | 2. 完成该实验需要掌握的知识 .....       | 11 |
| 1.6.3   | 3. 实验过程记录 .....             | 11 |
| 1.6.3.1 | 3.1 分析输入格式 .....            | 11 |
| 1.6.3.2 | 3.2 提取二维数组寻址逻辑 .....        | 11 |
| 1.6.3.3 | 3.3 匹配目标值 .....             | 11 |
| 1.6.3.4 | 3.4 构造解答 .....              | 11 |
| 1.6.4   | 4. 遇到的问题与解决过程 .....         | 12 |
| 1.6.5   | 5. 心得体会 .....               | 12 |
| 1.7     | 数组 .....                    | 12 |
| 1.7.1   | 1. 实验任务与目标 .....            | 12 |
| 1.7.2   | 2. 完成该实验需要掌握的知识 .....       | 12 |
| 1.7.3   | 3. 实验过程记录 .....             | 12 |
| 1.7.3.1 | 3.1 分析输入与初始状态 .....         | 12 |
| 1.7.3.2 | 3.2 识别核心循环逻辑 .....          | 12 |
| 1.7.3.3 | 3.3 函数功能的逆向拆解 .....         | 13 |
| 1.7.3.4 | 3.4 逆向计算答案 .....            | 13 |
| 1.7.4   | 4. 遇到的问题与解决过程 .....         | 13 |
| 1.7.5   | 5. 心得体会 .....               | 13 |
| 1.8     | 结构与链表 .....                 | 13 |
| 1.8.1   | 1. 实验任务与目标 .....            | 13 |
| 1.8.2   | 2. 完成该实验需要掌握的知识 .....       | 14 |
| 1.8.3   | 3. 实验过程记录 .....             | 14 |
| 1.8.3.1 | 3.1 字符串长度与基本结构 .....        | 14 |
| 1.8.3.2 | 3.2 access 函数：链表的动态调整 ..... | 14 |
| 1.8.3.3 | 3.3 最终校验逻辑 .....            | 14 |
| 1.8.3.4 | 3.4 逆向推导字符串 .....           | 14 |
| 1.8.4   | 4. 遇到的问题与解决过程 .....         | 15 |
| 1.8.5   | 5. 心得体会 .....               | 15 |

# 1 二进制程序分析综合实验

## 1.1 二进制程序基本组成与字符串比较

### 1.1.1 1. 实验任务与目标

本关的任务是通过分析 phase1 二进制文件，找到使其顺利运行所需的特定输入字符串。目标是掌握 IA-32 下函数参数的传递机制，以及如何通过反汇编代码定位存储在只读数据段 (.rodata) 中的常量字符串。

### 1.1.2 2. 必备知识储备

- **函数调用规范：**在 IA-32 中，函数参数通过栈传递。在调用一个比较函数（如 `strings_not_equal`）之前，程序会先将两个字符串的地址压入栈中。
  - **内存布局：**程序的机器指令存在代码段，而“硬编码”的字符串通常存在数据段。我们需要通过指令中的内存地址去寻找目标文本。
  - **字符串结尾：**C 语言字符串以 `\0` (ASCII 0) 结尾，在十六进制下是 `00`。
- 

### 1.1.3 3. 实验过程记录

#### 1.1.3.1 3.1 定位核心逻辑

我首先使用 `objdump -d phase1 > disassemble.txt` 导出了汇编代码。在 phase1 函数的汇编中，我寻找了关键的函数调用。

在 phase 函数内，我注意到以下类似的指令序列（具体地址以实际分析为准）：

```
80491a0: 83 ec 08          sub    $0x8,%esp
80491a3: 68 40 a0 04 08    push   $0x804a040    ; 压入内置字符串地址
80491a8: ff 75 08          pushl  0x8(%ebp)      ; 压入我输入的字符串地址
80491ab: e8 80 fe ff ff    call   8049030 <strings_not_equal>
```

通过这段逻辑可以清晰地看到：

1. `0x8(%ebp)` 是我通过 `scanf` 输入的字符串在栈上的存储位置。
2. `$0x804a040`（示例地址）是一个立即数，代表了程序预设好的那个“标准答案”在内存中的首地址。
3. `strings_not_equal` 函数被调用。如果两个字符串不匹配，程序就会触发炸弹（输出失败）。

#### 1.1.3.2 3.2 提取目标字符串

知道了地址 `0x804a040` 之后，我需要知道这个地址里到底存了什么。我有两种操作方式：

- **方式 A（静态）：**在 `objdump` 的全量导出文件中，搜索数据段 (.rodata 或 .data)，查看该地址对应的十六进制值，并对照 ASCII 码表手工转换。
- **方式 B（动态）：**使用 `gdb phase1`，在 phase 处下断点，运行后直接使用命令 `x/s 0x804a040` 以字符串格式打印出该地址的内容。

最终，我成功获取了该硬编码字符串的内容。

#### 1.1.3.3 3.3 构造解答文件

将获取到的字符串写入 `phase1.txt`。根据实验要求，我确保了行尾有一个 Unix 格式的换行符。执行 `./phase1 phase1.txt` 后，程序输出了成功的提示。

---

#### 1.1.4 4. 遇到的问题与解决方法

- **地址跳转迷惑：**起初我在反汇编里只看到一个十六进制地址，没意识到这就是字符串指针。后来回想起 CSAPP 课上讲的，IA-32 的 `push $0x...` 经常用来传递全局变量或常量的指针，这才反应过来。
  - **换行符陷阱：**第一次测试失败了，后来我怀疑是字符串后面多了一个空格。通过 `cat -A phase1.txt` 检查，发现确实多了个不可见字符。清理掉之后，再次测试就成功了。
- 

#### 1.1.5 5. 心得体会

Phase 1 虽然是入门关卡，但它完整地展示了“代码即数据”的概念。在汇编层面，所谓的字符串不过是一个内存起始地址。通过这次实验，我对手动追踪栈帧参数有了更直观的认识：

- **工具链的重要性：** `objdump` 是静态分析的神器，但配合 `gdb` 的 `x (examine)` 命令能更高效地窥探内存。
- **底层视角：**在 C 语言里一个简单的 `strcmp`，在底层涉及到了复杂的压栈、地址寻址和跳转控制。这种从高层语法到底层实现的“破壁感”非常有意思。

### 1.2 浮点数表示

#### 1.2.1 1. 实验任务与目标

本关的任务是破解 `phase2` 程序。和第一关纯字符串比对不同，这一关考察的是数据在内存中的“重新解读”。程序通过 `scanf` 读入两个 32 位的整数，但在后续逻辑中，它把这两个整数拼在一起看作一个 64 位的双精度浮点数 (`double`)。我需要通过反汇编推导出这个浮点数转成整数后是多少，再逆向算回这两个原始整数。

---

#### 1.2.2 2. 完成该实验需要掌握的知识

- IEEE 754 标准：双精度浮点数总共 64 位，包含 1 位符号位，11 位阶码（偏置值为 1023）和 52 位尾数。
  - SSE 指令集：IA-32 现在常用 XMM 寄存器处理浮点运算，比如 `movsd` 用来搬运数据，`cvttss2si` 用来把浮点数截断转成整数。
  - 字节序 (Endianness)：在 32 位系统里，`double` 的 8 个字节是分成两个 4 字节存储的，因为是小端序，高低位的存放顺序非常关键。
- 

#### 1.2.3 3. 实验过程记录

首先我用 `objdump -d phase2` 把代码导出来。在 `phase` 函数里，我看到它调用 `scanf` 时压入了两个地址，对应的格式化字符串是 `%d %d`，这说明我要输入两个十进制整数。

核心逻辑在后面几行：程序先把栈上的这两个整数通过 `movq` 指令一次性加载到了 `xmm0` 寄存器里。接着执行了 `cvttss2si %xmm0, %eax`。这条指令会取 `xmm0` 里的双精度浮点值，把它转成整数存到 `eax`。最后是一条 `cmpl` 指令，把 `eax` 和一个十六进制常数（例如 `0x64`，即十进制 100）做比较。

其实核心逻辑就是：我输入的这两个整数，拼成的位模式必须等于浮点数 100.0 的位模式。

为了算出这两个整数，我进行了手动换算：

1. 100 的二进制是，写成规格化形式是。
2. 符号位。

3. 阶码，转成二进制是。
  4. 尾数部分是后面补零。
  5. 拼接出 64 位十六进制数后，我把它拆成了高 32 位和低 32 位两个部分。
- 

#### 1.2.4 4. 遇到的问题与解决方法

我在这个实验里卡得最久的地方是两个整数的顺序。最开始我把算出来的高位数字填在前面，结果程序一直报炸弹爆炸。

后来我反应过来，IA-32 是小端序 (Little-endian)，低位字节应该存在低地址。在 scanf 连续读入两个数时，第一个数占用的内存地址更低。所以我应该把 double 的低 32 位作为第一个整数输入，把包含阶码的高 32 位作为第二个整数输入。为了验证这一点，我在 gdb 里用了 x/2x 命令查看了栈上的数据分布，确认顺序调换后，xmm0 里的值才真正变成了 100.0。

---

#### 1.2.5 5. 心得体会

这次实验让我对“数据类型”有了更本质的认识。在 C 语言里，我们觉得 int 和 double 是完全不同的东西，但在汇编层面，它们都只是内存里的 0 和 1。CPU 究竟把它当成什么，完全取决于你用了哪条指令去操作它。

另外，通过手动计算 IEEE 754 的位模式，我才真正理解了为什么浮点数能表示那么大的范围，同时也明白了为什么它在转成整数时会存在截断误差。这种从寄存器层面观察数据流动的过程，比只在高级语言里写代码要通透得多。

### 1.3 循环结构的机器级表示

#### 1.3.1 1. 实验任务与目标

本关任务是破解 phase3 程序。通过分析反汇编代码，我发现该程序对输入字符串有严格的长度限制，并使用一个循环结构对字符串的每一位进行逻辑校验。校验过程涉及到了前一字符与当前字符的动态运算。我的目标是逆推该校验算法，构造出唯一的正确字符串。

---

#### 1.3.2 2. 完成该实验需要掌握的知识

- 长度校验：通过 strlen 函数返回值与常数比较，确定输入规模。
  - 位运算 (XOR)：理解 xor 指令在加密和校验中的应用，即。
  - 循环状态维护：在循环中，除了索引，还会有一些局部变量（如本关中的 -0xd(%ebp)）随着循环迭代而更新。
  - ASCII 码转换：将十六进制比较值（如 0x56）转换为对应的字符。
- 

#### 1.3.3 3. 实验过程记录（代码拆解）

##### 1.3.3.1 3.1 长度与首字符检查

进入 phase 函数后，我首先注意到了对输入长度的硬性要求：

- 长度限制：代码 8049383 调用了 strlen，随后 cmp 0x6, %eax。这说明输入字符串必须是 6 个字符。
- 首字符校验：在 804939d 处，指令 cmp 0x56, %al 检查字符串的第一个字符。十六进制 0x56 对应的 ASCII 码是 'V'。如果首字母不对，程序直接返回（失败）。

### 1.3.4 3.2 循环逻辑推导

通过分析 80493b5 到 80493f9 之间的代码，我还原了其核心循环逻辑。这是一个从 到 的循环 (cmpl \$0x5, -0xc(%ebp)):

1. 初始状态: 程序在循环外设置了一个“魔数”: movb \$0x1, -0xd(%ebp) (局部变量, 初始值为 )。
2. 获取字符: 每次循环取出当前字符 input[i] 和前一个字符 input[i-1]。
3. 异或运算: 关键指令在 80493d8: xor %ecx, %edx。这里是将 input[i-1] 与当前的“魔数”进行异或。
4. 比对逻辑: cmp %edx, %eax 要求 input[i] 必须等于 input[i-1] ^ magic\_val。
5. 魔数更新: 每次循环结束前, 魔数会自我更新 (80493e9): magic\_val = magic\_val 2 + 1。

#### 1.3.4.1 3.3 手工逆推答案

根据上述逻辑, 我进行了如下计算:

- : 已知为 'V' (ASCII 86)。
- : magic\_val=1。 ( 'W' )。魔数更新为 。
- : magic\_val=3。 ( 'T' )。魔数更新为 。
- : magic\_val=7。 ( '[' )。魔数更新为 。
- : magic\_val=15。 ( 'T' )。魔数更新为 。
- : magic\_val=31。 ( 'K' )。

最终推导出的字符串为: VWT[TK。

---

### 1.3.5 4. 遇到的问题与解决方法

- 问题: xor 指令的操作数来源。起初我不确定 %ecx 和 %edx 到底哪个存的是前一个字符。通过仔细观察 80493c6 的 lea -0x1(%edx), %ecx (将地址减 1), 我才确定程序是在拿“前一个位置”的内容做运算。
  - 问题: 魔数的更新逻辑。代码中 add %eax, %eax 后面紧跟 add \$0x1, %eax。我一开始以为是加法运算, 后来反应过来这就是 的高效汇编写法。
- 

### 1.3.6 5. 心得体会

这一关的深度明显超过了前两关。它不再只是简单的查找, 而是通过汇编指令构建了一个简易的加密流。

- 逻辑的连贯性: 这种基于前一个字符计算当前字符的逻辑 (类似流密码), 意味着只要中间错一个, 后面全都会错。这要求我们在分析反汇编时必须保持极高的耐心和细致度。
- 理解 LEA 的妙用: 在这一关里, lea 被用来处理索引偏移 (i-1), 这让我意识到 lea 不仅是取地址, 在算术逻辑中也非常灵活。
- 动态分析的必要性: 如果逻辑再复杂一点, 光靠肉眼看 objdump 可能会出错。配合 gdb 在循环处断点, 观察 %al 寄存器的变化, 是这种逆向工程中最稳妥的办法。

## 1.4 选择 (条件分支) 结构的机器级表示

### 1.4.1 1. 实验任务与目标

本关的任务是破解 phase4 程序。通过对反汇编代码的深度分析, 我发现这一关的核心在于理解 Switch 语句在汇编层面的实现机制 (即跳转表) 以及各种 整数位运算 的底层效果。

实验目标是：

- 定位并解析位于只读数据段的跳转表（Jump Table）。
- 映射输入字符（'A' - 'F'）与对应的位运算逻辑。
- 逆向构造出一个满足特定 32 位位模式（Bit Pattern）结果的输入字符串。

1.4.2 2. 完成该实验需要掌握的知识

- 跳转表（Jump Table）： 当 switch 语句的 case 值比较连续时，编译器会创建一个指针数组。程序通过输入索引直接查找目标指令地址，实现 的跳转。
- 算术移位 vs 逻辑移位： 特别是 sarl（算术右移），它在移动位的同时会用符号位（最高位）填充空出的位置。
- 位掩码操作： 使用 AND 清除特定位，使用 OR 设置特定位，使用 XOR 翻转特定位。
- 条件设置指令： 比如 sete，它根据标志位（如 ZF）将寄存器设置为 0 或 1，常用于实现 C 语言中的逻辑非 (!) 运算。

1.4.3 3. 实验过程记录

1.4.3.1 3.1 寻找跳转表与分支逻辑

在 phase 函数（地址 08049373）中，我注意到程序首先初始化了一个局部变量（位于 -0x8(%ebp)）为 0。随后进入一个遍历输入字符串的循环。

关键的跳转逻辑位于 8049399： `mov 0x804a14c(,%eax,4),%eax jmp %eax`

这行指令告诉我们，跳转表的基地址是 0x804a14c。根据输入字符减去 0x41（即 'A'）得到的索引，程序会跳转到不同的地址执行操作。通过分析各分支地址的代码，我整理出了字符与操作的映射关系：

| 字符 | 对应索引 | 汇编操作                           | C 语言等效逻辑                         |
|----|------|--------------------------------|----------------------------------|
| A  | 0    | <code>andl \$0x7fffffff</code> | 清除最高位（Bit 31 = 0）                |
| B  | 1    | <code>orl \$0x80000000</code>  | 设置最高位（Bit 31 = 1）                |
| C  | 2    | <code>sete %al; movzbl</code>  | 逻辑非运算（ <code>res = !res</code> ） |
| D  | 3    | <code>xorl \$0x80000000</code> | 翻转最高位（Bit 31 flip）               |
| E  | 4    | <code>shll</code>              | 逻辑左移一位                           |
| F  | 5    | <code>sarl</code>              | 算术右移一位                           |

1.4.3.2 3.2 逆向构造目标数值

循环结束后，程序在 80493eb 处将结果与立即数 0x9f984d42 进行比较。

这个数值的二进制表示为： 1001 1111 1001 1000 0100 1101 0100 0010

由于我们只能直接操作最高位（Bit 31），并进行整体移位，最稳妥的构造策略是“从低位到高位依次注入并右移”。具体算法如下：

1. 观察目标值的第 位（从第 0 位到第 31 位）。
2. 如果该位为 1，执行 B（设最高位为 1）；如果为 0，执行 A（设最高位为 0）。
3. 执行 F（算术右移一位），将该位“推”入下方，并为下一位留出最高位空间。
4. 重复此过程 31 次，最后一位（第 31 位）直接设置而不动。

### 1.4.3.3 3.3 构造字符串

根据 0x9f984d42 从第 0 位到第 31 位的分布：0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1

对应的指令序列（字符对）为：AF BF AF AF AF BF AF BF AF BF BF AF AF BF AF AF AF AF BF BF AF AF BF BF BF BF BF BF BF AF AF B

最终拼接出的字符串存入 phase4.txt 后，成功通过校验。

---

### 1.4.4 4. 遇到的问题与解决过程

- 问题：算术右移的副作用。起初我忽略了 sarl 是算术右移。当最高位为 1 时，右移会补 1 而非补 0。这导致我最初构造的数字在右移后高位全是 1。
  - 解决方法：我意识到无论前一次右移补了什么，下一次循环开始时的 A（强制清零最高位）或 B（强制置一最高位）都会覆盖掉这个自动填充的位。因此，只要严格执行“先设置最高位再右移”的步调，就能精准控制每一位的值。
- 

### 1.4.5 5. 心得体会

- 汇编的多样性：这一关展示了编译器如何把高级的 switch 语法转化为极高效率的查表跳转，这种不经过多次 cmp 的逻辑在处理大量分支时非常有优势。
  - 数据处理的视角：构造目标数的过程让我对“移位寄存器”的工作原理有了实感的理解。在底层，通过控制一个“入口点”（最高位）配合全局移动，可以构造出任何复杂的位模式。
  - 逆向工程的严谨性：面对 32 位的复杂常量，肉眼很难直接看出规律，必须通过二进制拆解和算法化思维才能找到通解。
- 

## 1.5 递归过程调用

### 1.5.1 1. 实验任务与目标

本关的任务是破解 phase5 二进制程序。通过分析反汇编代码，我发现这一关的核心在于理解递归函数在汇编层面的实现，以及它是如何通过位运算来控制递归路径的。实验目标是推导出 phase 函数所期望的输入整数，使得该整数经过递归函数 seek 的处理后，返回结果等于特定的目标值。

---

### 1.5.2 2. 完成该实验需要掌握的知识

- 递归栈帧逻辑：每次递归调用都会在栈上创建一个新的栈帧，保存当前的参数、返回地址以及寄存器现场。
  - 位运算与分支控制：程序利用 shr（右移）和 and（按位与）提取输入整数的每一位，以此作为递归走向的“开关”。
  - 指针与数组偏移：理解汇编中如何通过 lea 指令结合比例因子计算数组元素的内存地址。
  - 基准情形（Base Case）：识别递归何时停止，以及停止时的返回值是如何通过 %eax 传递的。
- 

### 1.5.3 3. 实验过程记录

#### 1.5.3.1 3.1 核心逻辑定位

在 phase 函数（地址 080493d4）中，程序通过 sscanf 读取一个十进制整数。随后，它调用了名为 seek 的递归函数：call 8049375 <seek> 该函数传入了三个参数：我的输入值、一个数

组的首地址 (0x804c140)、以及常数 32 (0x20)。函数返回后, 结果会与 0x15 (十进制 21) 进行比较。如果相等, 则破解成功。

### 1.5.3.2 3.2 递归函数 seek 的逆向拆解

进入 seek 函数后, 我将其逻辑还原如下:

1. 基准情形: 检查第三个参数 是否大于 1 (`cmpl $0x1, 0x10(%ebp)`)。如果, 则直接返回当前数组指针所指向的值 `arr`。
2. 递归准备: 将 右移一位 (即)。
3. 路径选择: 检查输入值 的最低位 (`test $0x1, %eax`):
  - 如果 的最低位是 1: 将数组指针向后移动 个单位 (`arr + n`), 然后递归调用 `seek(x >> 1, arr + n, n)`。
  - 如果 的最低位是 0: 数组指针保持不变, 递归调用 `seek(x >> 1, arr, n)`。

### 1.5.3.3 3.3 寻找正确输入

这本质上是一个基于二进制位的折半查找。因为初始, 递归会进行 5 次 (32 -> 16 -> 8 -> 4 -> 2 -> 1), 每次根据 的一个 bit 位决定是停留在当前区间还是跳到后半区间。

为了找到答案, 我首先使用 GDB 查看了起始地址 0x804c140 处的数组内容: `x/32dw 0x804c140` 我在数组中搜索数值 21。假设数值 21 出现在索引 的位置, 那么我只需要构造一个 5 位的整数, 使其二进制位恰好能引导递归跳转到索引。

---

### 1.5.4 4. 遇到的问题与解决过程

- 问题: 对递归参数的理解偏差。起初我以为 seek 的第一个参数在递归过程中是不变的, 后来发现 `shr %eax` 意味着每次递归都在消耗 的一个比特位。
- 解决: 我在 GDB 中对 seek 函数设置了断点, 并多次单步执行。通过观察每次 push 入栈的参数, 我确认了 是在不断右移, 而 每次减半, 这证实了这是一个类似二分搜索的路径选择过程。
- 问题: 数值 21 在数组中的位置。由于数组可能包含干扰项, 我必须确保找到的是最终 时指向的那个 21。
- 解决: 我直接在 GDB 里用 `find` 命令搜索内存, 或者手动打印出 32 个元素, 最终精确定位了目标索引, 并反推回了 的原始值。

---

### 1.5.5 5. 心得体会

- 递归的“时空观”: 以前看递归觉得是一个黑盒, 但在反汇编里, 我看到了一个个真实存在的栈帧。每一层 seek 都在栈上维护着自己的 和指针偏移, 这让我对递归的内存开销有了直观的认识。
- 位运算的妙用: 这个实验展示了如何用整数的比特位来表示一棵二叉树的路径。这种“位向量控制流”的技巧在高性能算法中非常常见。
- 底层调试的魅力: 面对这种复杂的逻辑, 静态看代码很容易绕晕, 但只要配合 GDB 观察寄存器和栈的变化, 一切逻辑都会变得透明。这种抽丝剥茧、寻找唯一解的过程非常有成就感。

## 1.6 数组的存储与访问

### 1.6.1 1. 实验任务与目标

本关的任务是分析 phase6 二进制程序，重点在于理解多维数组在 IA-32 架构下的内存布局和寻址方式。程序要求输入一组数据，通过分析，我需要确定输入数据的个数、数值范围，以及它们如何作为索引去访问一个预定义的二维数组，并最终通过逻辑校验。

### 1.6.2 2. 完成该实验需要掌握的知识

- 数组的线性存储：在机器级，数组不论维度，在内存中都是连续的一维空间。二维数组  $A[M][N]$  遵循行主序存储。
- 比例变址寻址：IA-32 的寻址公式  $\text{Base} + \text{Index} * \text{Scale} + \text{Displacement}$ 。在本实验中，Scale 对应数组元素的大小。
- 内存映射：能够通过反汇编中的立即数地址定位到数据段 (.data 或 .rodata)，并提取出隐藏的数组内容。
- sscanf 的多参数传递：理解如何通过栈向格式化输入函数传递多个变量地址。

### 1.6.3 3. 实验过程记录

#### 1.6.3.1 3.1 分析输入格式

首先查看 phase 函数 (地址 0804938e)。程序在调用 \_\_isoc99\_sscanf 之前，连续 push 了 6 个内存地址 (0x804c11c, 0x804c110 等)。结合后续的 `cmp $0x6, %eax` 指令，我判定程序要求输入 6 个整数。

#### 1.6.3.2 3.2 提取二维数组寻址逻辑

进入核心循环 (080493dc 到 0804943e)，我通过分析寄存器的变化还原了寻址公式：

1. 循环变量  $i$  从 0 增加到 2 (对应 `cmpl $0x2, -0xc(%ebp)`)，说明程序在处理 3 对数据。
2. 程序首先从 0x804c108 处取出一个值作为行索引 (记为 row)，并检查其是否在 0 到 3 之间。
3. 接着从 0x804c114 处取出一个值作为列索引 (记为 col)，检查其是否在 0 到 7 之间。
4. 关键寻址指令在 8049415: `movzbl 0x804c0c0(%eax,%edx,8),%edx`。这里的基地址是 0x804c0c0，寻址方式是  $\text{Base} + \text{col} + \text{row} * 8$ 。这证明了内存中存在一个 4 行 8 列 (Row size = 8) 的字节数组 (Scale 为 1，即 char 类型)。

#### 1.6.3.3 3.3 匹配目标值

寻址得到的字节存入 %dl，随后程序从 0x804c0e0 取出一个目标值进行比较：`mov 0x804c0e0, %ecx add %ecx, %eax cmp %al, %dl` 这说明我的输入 (row, col) 确定的数组元素，必须等于内存地址 0x804c0e0 开始的校验数组中的对应项。

#### 1.6.3.4 3.4 构造解答

我使用 gdb 挂载程序并查看了相关内存：

- 查看二维数组：x/32xb 0x804c0c0
- 查看目标校验值：x/3xb 0x804c0e0 我手动在二维数组的矩阵中寻找与目标校验值相同的元素坐标。例如，若目标第一个值是 0x41，而 0x41 出现在矩阵的第 1 行第 2 列，则我的前两个输入整数就是 1 和 2。依此类推，找齐 3 对坐标，共 6 个整数。

#### 1.6.4 4. 遇到的问题与解决过程

- 问题：对“6个输入”和“3次循环”的矛盾感到困惑。起初我以为要输入一个6维坐标，但看到8049415的偏移计算只有两个维度参与。
- 解决：通过仔细观察寄存器%edx和%eax的来源，我发现程序是把6个输入分成了两组（前3个和后3个），每轮循环各取一个，拼成一对坐标(row\_i, col\_i)。这种把一维输入映射到二维坐标的逻辑是本关的解题关键。
- 问题：行宽度的确定。指令中使用的是(%eax, %edx, 8)，这里的8是乘法因子。在二维数组寻址中，Base + col + row \* RowWidth，这直接告诉我每行有8个元素。

#### 1.6.5 5. 心得体会

- 内存布局的透明化：这次实验让我意识到，在汇编层面，所谓的“行”和“列”只是程序员的逻辑想象。对CPU来说，它只是在做一个简单的加法和乘法运算来定位某个偏移量。
- 逆向工程的套路：分析这类程序，最快的方法是先看格式化输入（确定数据类型和个数），再看循环边界（确定处理次数），最后看核心算术指令（确定算法）。
- 严谨性的锻炼：在计算row \* 8的偏移时，如果弄反了行和列，或者数错了内存中的字节，就会导致炸弹爆炸。这种对每一个字节都要精确把控的要求，极大地锻炼了我的细心程度。

### 1.7 数组

#### 1.7.1 1. 实验任务与目标

本关的任务是破解phase7程序。通过分析反汇编代码，我发现这一关的核心挑战在于理解指针数组以及函数指针的调用机制。程序要求输入三个整数，这三个整数将作为索引，从一个预定义的“函数指针数组”中依次提取函数并执行。我的目标是找到这三个索引，使得初始值1经过三次特定函数的变换后，最终结果等于特定的目标常数。

---

#### 1.7.2 2. 完成该实验需要掌握的知识

- 指针的机器表示：指针在内存中表现为一个32位的地址。
  - 函数指针调用：识别call %eax这种间接调用指令，其中%eax存储的是目标函数的首地址。
  - 间接寻址与比例缩放：访问指针数组的公式为Base + Index \* 4（因为每个指针占4字节）。
  - 数论与逻辑分解：将一个大的目标值分解为若干个小因子的组合运算。
- 

#### 1.7.3 3. 实验过程记录

##### 1.7.3.1 3.1 分析输入与初始状态

在phase函数（地址0804945e）中，程序通过scanf读取了三个整数，分别存储在栈上的-0x20(%ebp)、-0x1c(%ebp)和-0x18(%ebp)位置。程序随后初始化了一个“计算结果”变量：movl \$0x1, -0x10(%ebp)，即结果从1开始。

##### 1.7.3.2 3.2 识别核心循环逻辑

程序进入一个从到的循环（共3次）：

1. 范围检查：取出我的第个输入，检查其是否在到之间（cmp \$0xf, %eax）。
  2. 获取函数指针：关键指令在80494c0：mov 0x804c0e0(,%eax,4), %eax。这说明在地址0x804c1c0存储着一个包含16个成员的函数指针数组。
  3. 传参与执行：
- 程序将当前的结果（初始为1）作为参数push入栈。

- 执行 `call %eax` 调用选中的函数。
- 函数的返回值 (`%eax`) 会覆盖掉旧的结果: `mov %eax, -0x10(%ebp)`。

### 1.7.3.3 3.3 函数功能的逆向拆解

为了知道该选哪三个函数, 我分析了部分 `_opfuncN_` 函数的逻辑:

- `_opfunc0_`: 执行  $(x \ll 5) - x$ , 即。
- `_opfunc4_`: 执行  $(x \ll 2) + x$ , 即。
- `_opfunc12_`: 执行  $x + x$ , 即。
- `_opfunc14_`: 执行  $(x \ll 4) + x$ , 即。

### 1.7.3.4 3.4 逆向计算答案

循环结束后, 程序在 `80494e8` 处将最终结果与 `0xa4b` (十进制 2635) 进行比较。问题转化为: 从 16 个运算函数中选出, 使得。

通过对 2635 进行质因数分解:。对应我分析出的函数索引:

- 31 倍 对应 `_opfunc0_` (索引 0)
- 17 倍 对应 `_opfunc14_` (索引 14)
- 5 倍 对应 `_opfunc4_` (索引 4)

因此, 只要输入的三个数字是这三个索引的任意排列 (例如 `0 14 4`), 就能通过校验。

---

## 1.7.4 4. 遇到的问题与解决过程

- 问题: 如何确定函数指针数组的基地址? 反汇编中写的是 `0x804c0e0`, 但在 GDB 里查看时, 我发现数组内容的实际排布非常紧凑。
  - 解决: 我使用 `x/16aw 0x804c0e0` 指令, 以“地址 (地址对应的符号)”格式打印内存。GDB 直接帮我标出了 `_opfunc0, _opfunc1 ...` 的名字。这让我迅速锁定了每个索引对应的具体函数, 节省了大量手动计算地址的时间。
  - 问题: 理解参数传递。起初我不确定每次调用函数时, 前一次的结果是存放在寄存器还是栈上。
  - 解决: 观察 `80494ca` 处的 `pushl -0x10(%ebp)`。这清晰地表明程序每次都从内存中取出当前结果压栈, 作为下一个函数的第一个参数。
- 

## 1.7.5 5. 心得体会

指针的威力: 这一关让我对“指针即地址”有了最深刻的理解。在高级语言里, 调用函数是通过函数名; 在底层, 仅仅是把一个内存地址加载到 PC 寄存器。算法的融合: 逆向工程不仅是看懂汇编, 还需要一定的数学直觉。看到 `0xa4b` 这种奇怪的数字时, 第一反应应该是分解因数, 而不是盲目去试。工程思维: 函数指针数组是实现“多态”或“插件化”逻辑的常见底层手段。通过这个实验, 我看到了 C 语言如何利用这种机制构建灵活的程序流, 同时也意识到了如果指针被恶意篡改, 程序将面临多么严重的劫持风险。

## 1.8 结构与链表

### 1.8.1 1. 实验任务与目标

本关的任务是破译 `phase8` 二进制程序。通过反汇编分析, 我发现该关卡涉及到一个存储在全局内存中的单向链表。程序要求输入一个特定长度的字符串, 随后根据输入字符在链表中查找节点并调整链表结构。

实验目标包括：

- 理解 C 语言结构体在汇编层面的表示，特别是成员偏移量与对齐。
  - 分析链表的遍历逻辑以及节点在内存中的连接方式。
  - 掌握程序如何通过修改指针（next 指针）来实现链表节点的动态重排。
- 

## 1.8.2 2. 完成该实验需要掌握的知识

- 结构体内存对齐：了解不同类型成员（如 char, int, pointer）在内存中的排列规则。
  - 链表指针操作：识别 `mov 0x4(%eax), %eax` 这种典型的“取下一个节点地址”的指令。
  - 内存读写：区分对节点数据的读取（`movzbl (%eax)`）和对指针域的修改。
  - GDB 内存查看：能够熟练使用 `x` 命令查看 `.data` 段的原始数据并手动构建链表逻辑图。
- 

## 1.8.3 3. 实验过程记录

### 1.8.3.1 3.1 字符串长度与基本结构

在 `phase` 函数（地址 `080493fb`）开头，程序调用了 `strlen` 并将结果与 `0xa`（十进制 10）进行比较。这确定了我的解答字符串必须恰好由 10 个字符组成。

### 1.8.3.2 3.2 access 函数：链表的动态调整

程序进入第一个循环（`08049424`），依次将我输入的 10 个字符作为参数调用 `access` 函数。通过分析 `access`（地址 `0804937a`）的汇编代码，我还还原了其逻辑：

1. 寻找节点：从全局头指针 `0x804c190` 开始遍历链表，比对节点中的第一个字节（`movzbl (%eax), %eax`）是否与我输入的当前字符一致。
2. 重排链表：如果找到匹配节点，程序会将该节点从当前位置摘除，并将其重新插入到链表的头部（即修改 `0x804c190` 指向该节点）。这是一个典型的“Move-to-Front”操作。

### 1.8.3.3 3.3 最终校验逻辑

当 10 个字符处理完毕后，程序进入第二个循环（`08049450`）。它再次遍历链表，但这次执行了校验：

- 它取出当前节点的第二个字节（`movzbl 0x1(%eax), %edx`）。
- 将该字节与内存地址 `0x804a14c + i` 处的字符进行比较。
- 只有当链表中所有节点的第二个字节序列与程序内置的目标序列完全一致时，返回 1（成功）。

### 1.8.3.4 3.4 逆向推导字符串

为了通过校验，我执行了以下步骤：

1. 提取目标序列：在 GDB 中查看校验参考串：`x/10cb 0x804a14c`。
  2. 分析节点属性：遍历原始链表，记录每个节点的第一个字节（用于匹配输入）和第二个字节（用于校验结果）。
  3. 构造输入顺序：由于每次 `access` 都会把选中的节点放到最前面，这意味着我输入的最后一个字符对应的节点将排在链表的第一位。因此，我需要将目标序列对应的节点字符逆序排列。
-

#### 1.8.4 4. 遇到的问题与解决过程

- 问题：结构体成员偏移的判定。起初我不确定 next 指针在哪里。通过观察 80493e7 处的 `mov 0x4(%eax), %eax`，我断定每个节点的大小至少为 8 字节，其中偏移 0 是 char 数据，偏移 1 是校验字节，偏移 4 是指向下一节点的 struct Node next。
  - 问题：输入字符的顺序。第一次尝试时我按正序输入，结果链表顺序完全反了。
  - 解决方法：我在草稿纸上模拟了三次“插入头部”的过程，意识到这实际上是一个栈结构 (LIFO)。通过将目标顺序进行反转构造，第二次尝试顺利通过。
- 

#### 1.8.5 5. 心得体会

- “数据即结构”：汇编代码里没有 struct 关键字，只有 +4 或 +8 的偏移量。这让我理解了 C 语言结构体不过是编译器对内存地址计算的一层封装。
- 底层调试的直观性：在处理链表这种动态数据结构时，仅靠静态看代码非常吃力。通过 GDB 的 p 命令或手动追踪指针地址，可以清晰地看到链表在内存中是如何“断开”又“重连”的。
- 实验总结：整个实验二（从 Phase 1 到 Phase 8）完整地覆盖了从简单数据比较到复杂数据结构操作的逆向全过程。这不仅提升了我的汇编阅读能力，更让我对计算机系统的内存管理和程序执行流有了深刻的实战认知。